

AZ-204: Developing Solutions for Microsoft Azure

Day 1 — Azure Developer Foundations & App Service Deep Dive

INSTRUCTOR-LED TRAINING

AZ-204 CERTIFICATION



Day 1 Agenda

A structured journey from Azure fundamentals to hands-on App Service deployment.

01

Azure Developer Ecosystem

Portals, SDKs, CLI, and the developer mindset on Azure

02

ARM & Resource Groups

How Azure organizes and manages resources at scale

03

Compute Services Overview

App Service, Container Apps, AKS, and Azure Functions

04

Azure App Service Deep Dive

Web Apps, deployment methods, slots, scaling, and configuration

05

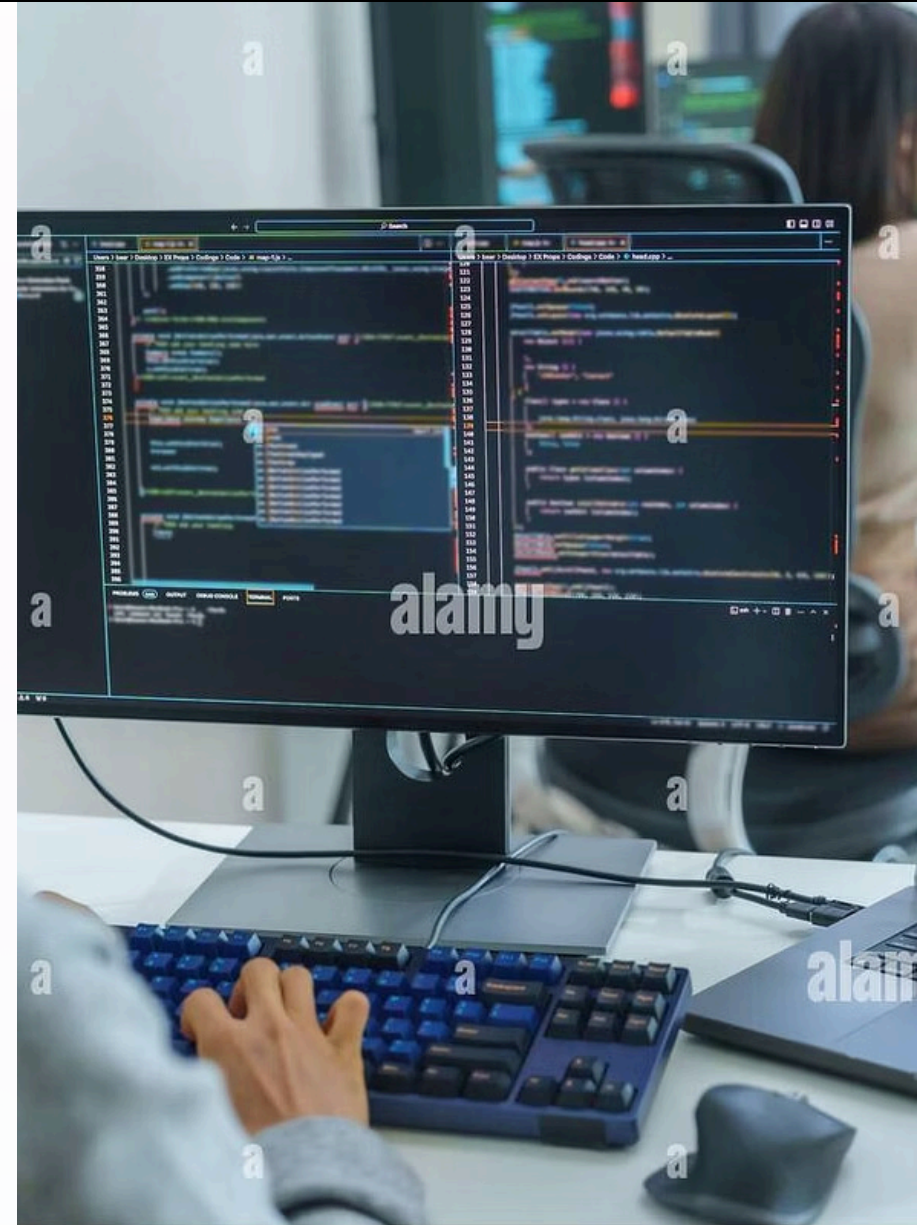
Day 1 Labs

Hands-on practice to reinforce the day's concepts

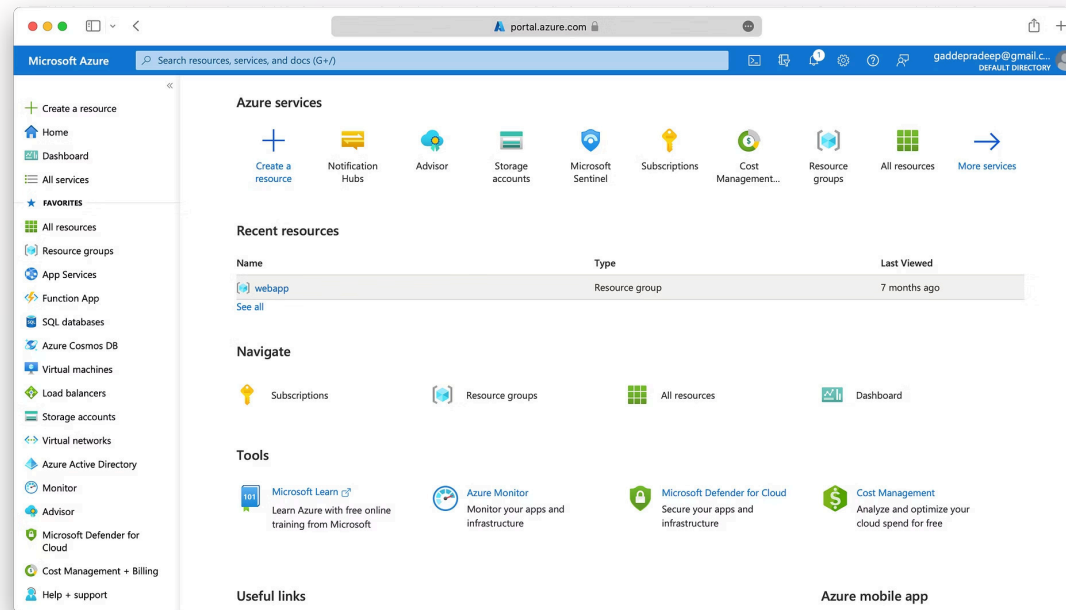
CHAPTER 1

The Azure Developer Ecosystem

Everything a developer needs to build, deploy, and manage cloud applications on Microsoft Azure.



What Is the Azure Developer Ecosystem?



Azure is Microsoft's cloud platform offering **200+ services** spanning compute, storage, networking, AI, and DevOps. Developers interact with it through multiple surfaces:

→ **Azure Portal**
Browser-based GUI for visual resource management

→ **Azure CLI / PowerShell**
Scriptable command-line tools for automation

→ **SDKs & REST APIs**
Programmatic access from .NET, Python, Java, JS, and more

Azure Developer Tools at a Glance



Azure Portal

Full-featured web UI. Great for exploration, monitoring, and one-off tasks.



Azure CLI

Cross-platform CLI for scripting, automation, and CI/CD pipelines.



Cloud Shell

Browser-based shell (Bash or PowerShell) pre-authenticated to your subscription.



Azure SDKs

Language-native libraries for .NET, Python, Java, Node.js, Go, and more.



VS Code Extension

Deploy and manage resources directly from your editor with Azure extensions.

Azure CLI & Cloud Shell

Azure CLI Essentials

- Installed locally or used via Cloud Shell
- Authenticate with `az login`
- Commands follow the pattern: `az [group] [command]`
- Supports output formats: JSON, table, TSV
- Scriptable — ideal for DevOps pipelines

 **Pro Tip:** Use `az configure --defaults group=myRG location=eastus` to set default context and reduce repetitive flags.

Common Developer Commands

```
# Login & set subscription
az login
az account set --subscription "My Sub"
```

```
# Create a resource group
az group create \
  --name myRG \
  --location eastus
```

```
# List web apps
az webapp list \
  --output table
```

Azure SDKs — Write Cloud Code in Your Language

Azure SDKs provide idiomatic, consistent access to Azure services. Each SDK follows the same **authenticate** → **client** → **operation** pattern.

.NET / C#

```
Azure.Identity,  
Azure.Storage.Blobs
```

Python

```
azure-identity, azure-  
storage-blob
```

JavaScript / Node

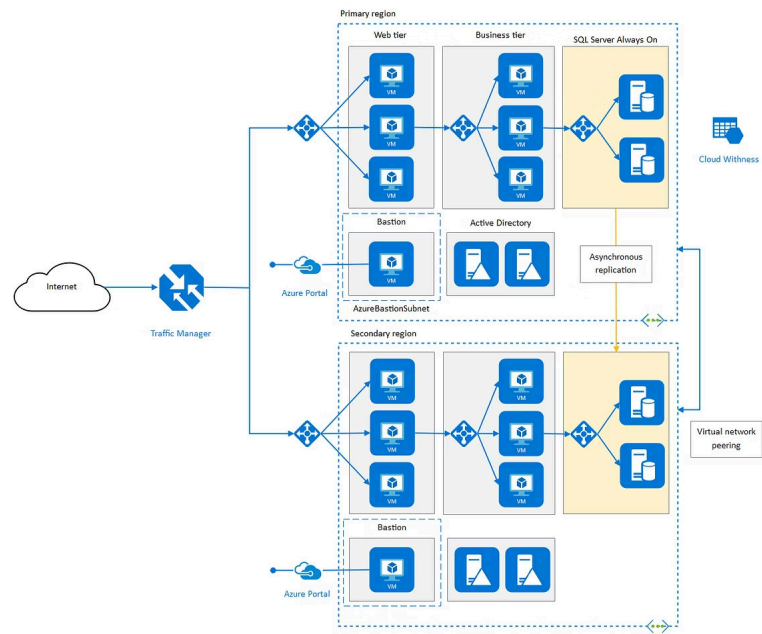
```
@azure/identity,  
@azure/storage-blob
```

Java

```
azure-identity, azure-  
storage-blob
```

SDK Authentication Pattern (Python)

```
from azure.identity import DefaultAzureCredential  
from azure.storage.blob import BlobServiceClient  
  
credential = DefaultAzureCredential()  
client = BlobServiceClient(  
    account_url="https://myacct.blob.core.windows.net",  
    credential=credential  
)
```



CHAPTER 2

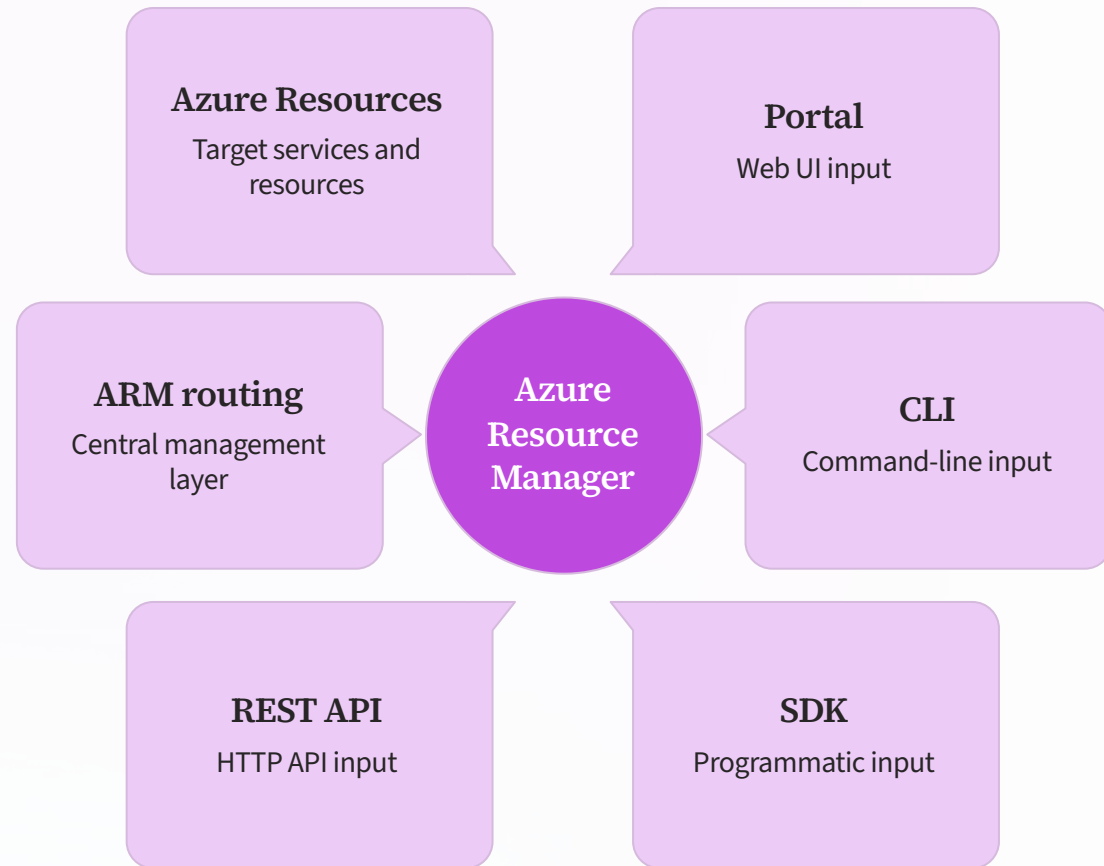
Azure Resource Manager & Resource Groups

The foundation of how Azure organizes, deploys, and manages every cloud resource.

Azure Resource Manager (ARM)

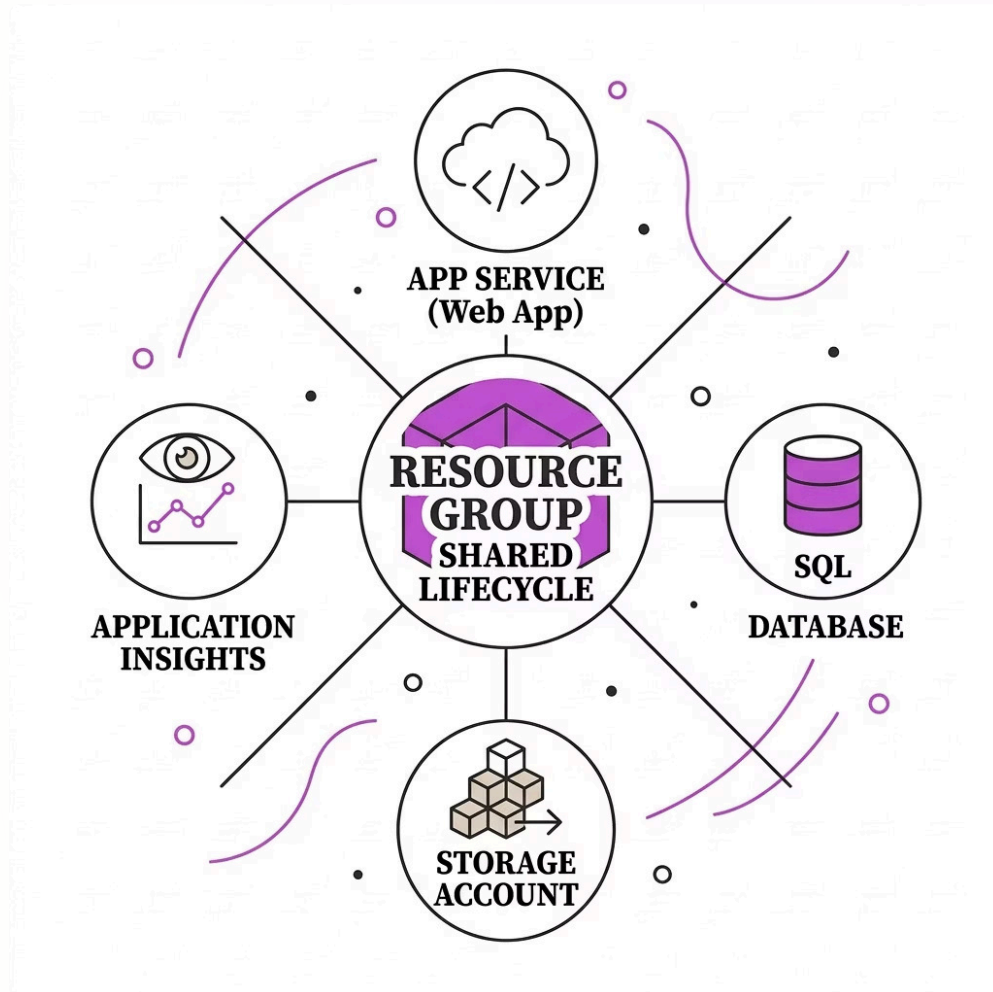
ARM is the **unified management layer** for all Azure services. Every action — whether from the Portal, CLI, SDK, or REST API — goes through ARM.

- Provides **consistent authentication** via Azure Active Directory
- Enforces **role-based access control (RBAC)**
- Supports **tags** for organization and cost tracking
- Enables **declarative deployments** via ARM templates or Bicep
- All operations are **auditable** via Activity Logs



All management traffic flows through ARM, giving a single control plane for the entire platform.

Resource Groups — Logical Containers for Azure Resources



Key Characteristics

- Every Azure resource **must** belong to exactly one resource group
- Resources in a group can be in **different regions**
- Deleting a resource group **deletes all resources** inside it
- Apply **RBAC and policies** at the group level for governance
- Use **tags** (e.g., environment=prod) for cost allocation

❏ **Best Practice:** Group resources by application lifecycle — dev, staging, and prod each get their own resource group.

ARM Templates & Bicep — Infrastructure as Code

Declare your desired Azure infrastructure in code — ARM deploys it consistently every time.

ARM Template (JSON)

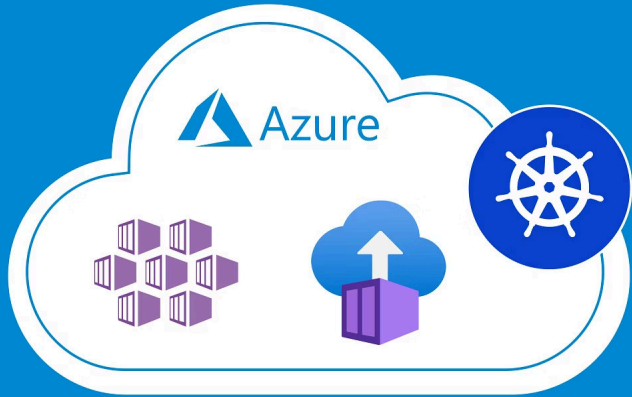
```
{
  "$schema": "...",
  "resources": [{
    "type": "Microsoft.Web/sites",
    "name": "myApp",
    "location": "[resourceGroup().location]",
    "properties": {
      "serverFarmId": "[resourceId(...)]"
    }
  }]
}
```

Bicep (Simplified Syntax)

```
resource appService 'Microsoft.Web/sites@2022-03-01' = {
  name: 'myApp'
  location: resourceGroup().location
  properties: {
    serverFarmId: plan.id
  }
}
```



Bicep is the modern, cleaner alternative to raw ARM JSON. It compiles to ARM templates and is the recommended IaC choice for Azure in 2024.



CHAPTER 3

Azure Compute Services Overview

Choosing the right compute model is one of the most important architectural decisions a developer makes on Azure.

Compute Services – Which Should I Use?

App Service

Fully managed PaaS for web apps, REST APIs, and mobile backends. No infrastructure management.

Container Apps

Serverless containers with built-in Dapr and KEDA scaling. No Kubernetes expertise needed.

AKS

Managed Kubernetes for teams that need full container orchestration control and flexibility.

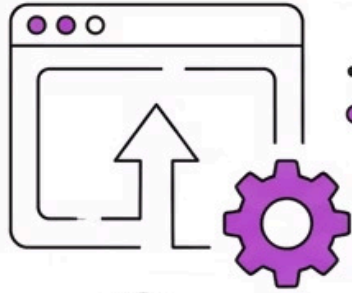
Azure Functions

Event-driven serverless compute. Pay only for execution time. Ideal for small, discrete workloads.

Rule of thumb: **App Service** for web apps → **Container Apps** for microservices → **AKS** for full orchestration → **Functions** for event-driven tasks.

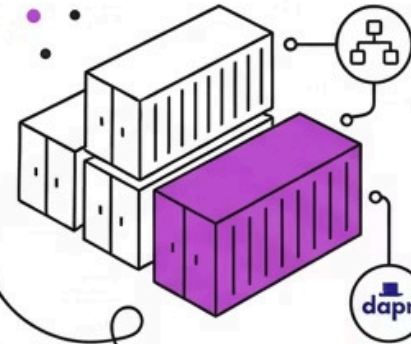
Compute Model Comparison

APP SERVICE



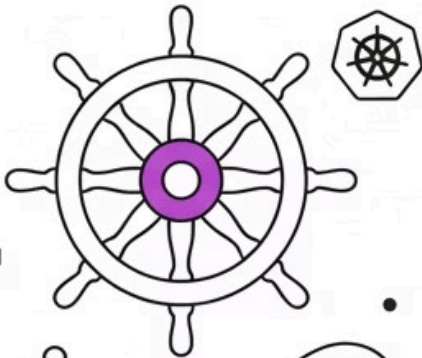
- PaaS
- Web Apps
- Auto-scale
- Easy Deploy

CONTAINER APPS



- Serverless Containers
- Microservices
- Dapr & KEDA

AKS



- Kubernetes
- Full Control
- Complex & Enterprise

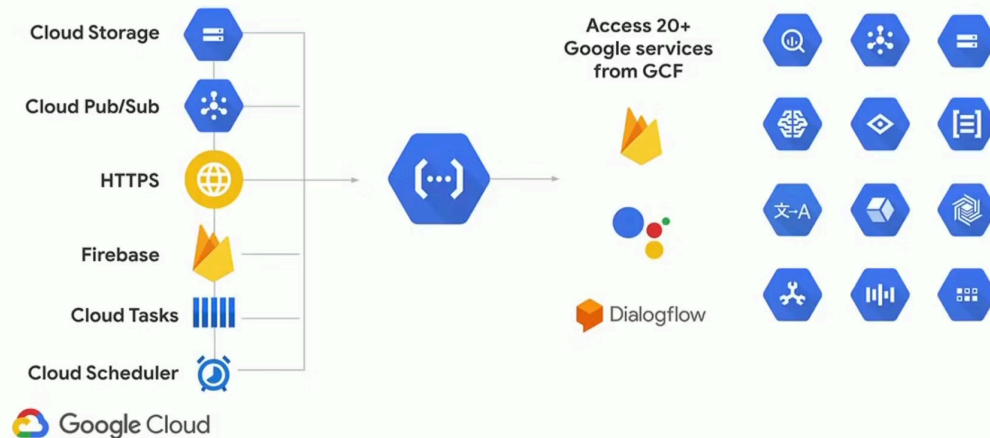
AZURE FUNCTIONS



- Serverless
- Event-driven
- Pay-per-execution
- Short Tasks

Azure Functions — Quick Introduction

Cloud Functions as cloud glue



What Are Azure Functions?

Small, single-purpose code blocks that run in response to **triggers** — no server provisioning required.

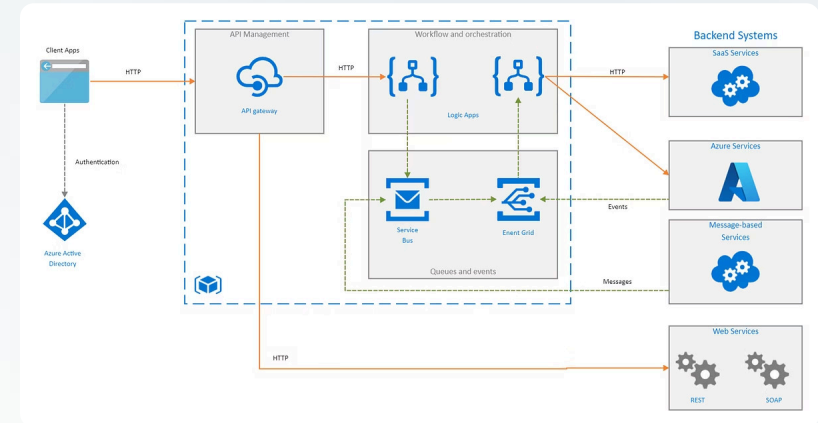
- **Triggers:** HTTP, Timer, Blob Storage, Service Bus, Event Grid
- **Bindings:** Declarative I/O connections — no boilerplate SDK code
- **Hosting plans:** Consumption (serverless), Premium, Dedicated
- **Languages:** C#, Python, JavaScript, Java, PowerShell

📌 We'll cover Azure Functions in depth on Day 3. Today's focus is App Service.

CHAPTER 4

Azure App Service Deep Dive

The go-to PaaS platform for hosting web applications, REST APIs, and mobile backends on Azure.



What Is Azure App Service?

A fully managed **Platform-as-a-Service (PaaS)** for building and hosting web applications. Azure handles OS patching, load balancing, and infrastructure.



Web Apps

Host ASP.NET, Node.js, Python, Java, PHP, Ruby applications



API Apps

Build and expose RESTful APIs with Swagger/OpenAPI support



Mobile Backends

Provide data sync, push notifications, and auth for mobile clients



WebJobs

Run background tasks and scripts alongside your web app

App Service Plans — The Underlying Infrastructure

Every App Service app runs on an **App Service Plan**, which defines the region, OS, compute size, and pricing tier.

Free / Shared (F1, D1)

Shared compute. Dev/test only. No SLA. Limited to 60 CPU minutes/day.

Basic (B1–B3)

Dedicated VMs. Manual scale. Good for low-traffic dev/test workloads.

Standard (S1–S3)

Auto-scale, deployment slots (up to 5), custom domains, SSL. Production-ready.

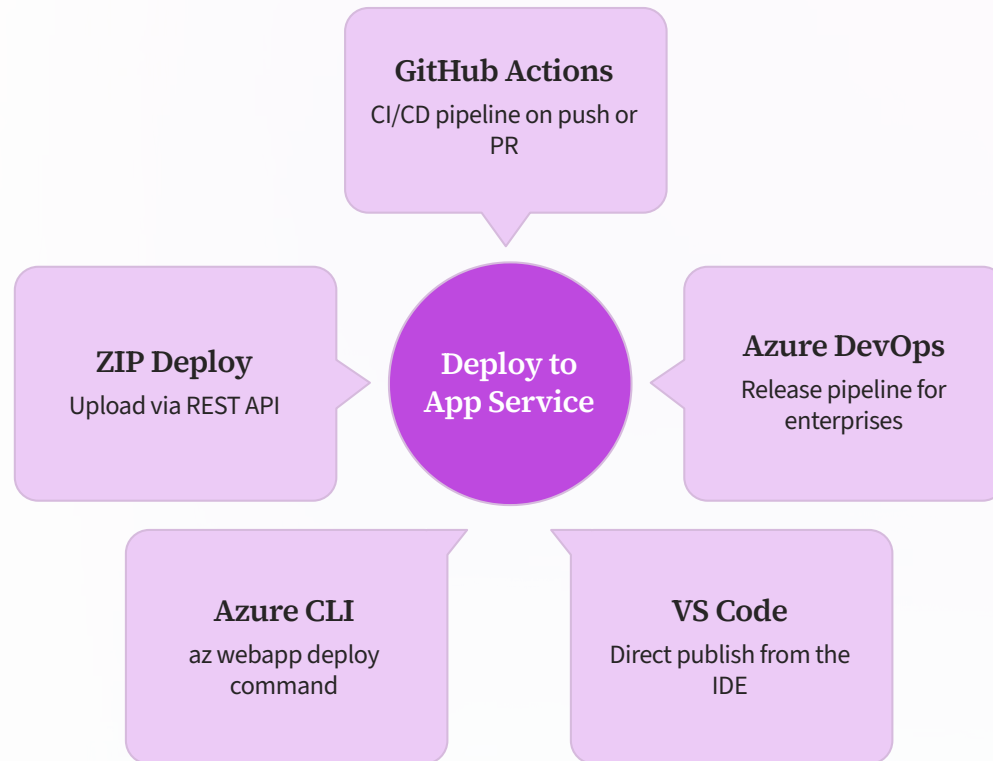
Premium (P1v3–P3v3)

Enhanced performance, VNet integration, more slots. High-traffic production workloads.

Isolated (I1v2–I3v2)

Dedicated environment (ASE). Max security, compliance, and network isolation.

Deploying to App Service — Your Options



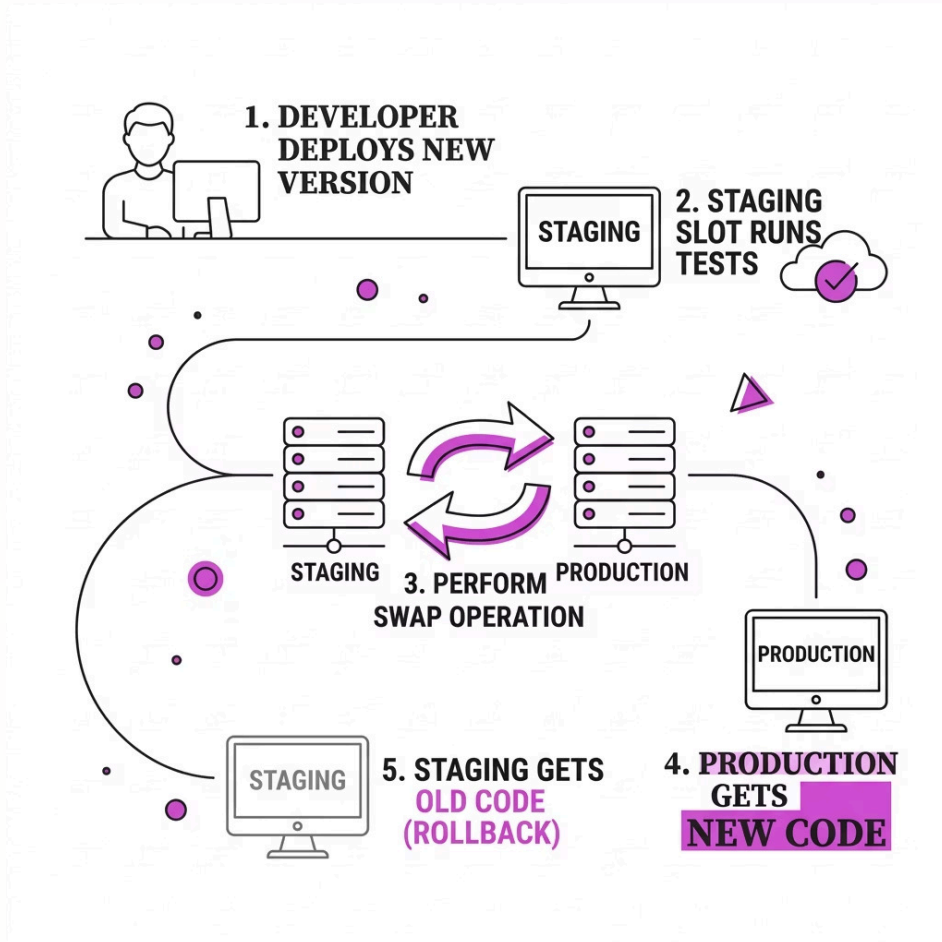
Common Deployment Methods

- GitHub Actions / Azure DevOps**
Automated CI/CD pipelines triggered on push or PR merge
- ZIP Deploy via CLI**
`az webapp deploy --src-path app.zip` — fast and scriptable
- VS Code Azure Extension**
Right-click deploy directly from your editor — great for development

Multiple deployment paths accommodate every team's workflow — from IDE to enterprise CI/CD.

Deployment Slots — Zero-Downtime Releases

Deployment slots are **live, independent environments** within the same App Service. Use them to test in production-like conditions before swapping.



How Slots Work

- Each slot has its **own URL**: `myapp-staging.azurewebsites.net`
- Slots share the same App Service Plan (no extra compute cost)
- **Swap** is instant — traffic reroutes with no downtime
- On swap, slot-specific settings remain in their slot
- Old production becomes staging — **instant rollback** capability

❏ Available on **Standard tier and above**. Standard allows up to 5 slots; Premium allows up to 20.

Scaling App Service Applications

Scale Up (Vertical)

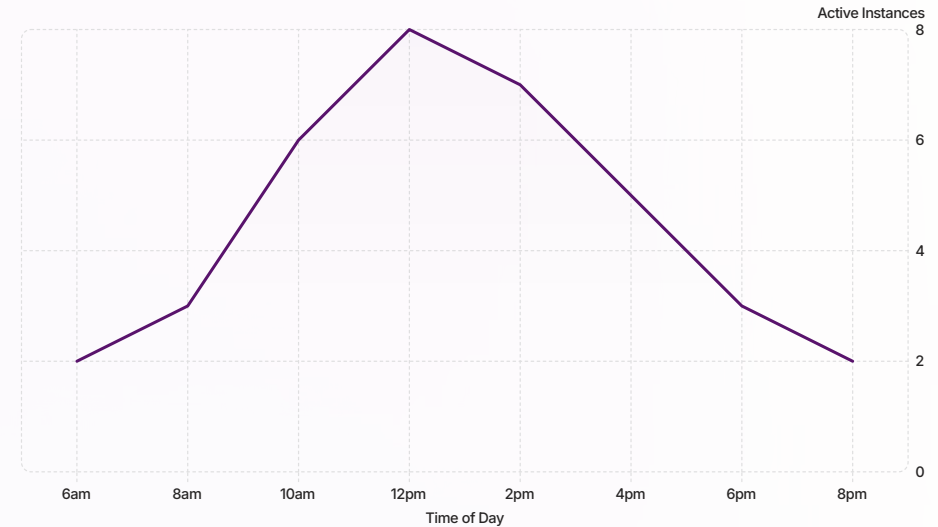
Move to a larger App Service Plan tier to get more CPU, RAM, and storage.

- Useful when app is memory or CPU-bound
 - Requires a brief restart — plan for maintenance windows
 - Example: B2 → S2 → P2v3
-

Scale Out (Horizontal)

Add more VM instances to distribute load across multiple workers.

- **Manual:** Set a fixed instance count
- **Auto-scale:** Rules based on CPU %, memory, HTTP queue length, or schedule
- Requires **Standard tier or above**



Auto-scale responds to traffic patterns, scaling out at peak hours and back in during off-peak periods to optimize cost.

Application Configuration & Environment Variables

App Service provides a secure, centralized way to manage configuration — no secrets in source code.

Configuration Sources

App Settings

Key-value pairs injected as **environment variables** at runtime. Override appsettings.json or .env files.

Connection Strings

Securely store DB connection strings. Type-aware (SQL, MySQL, Custom) for correct framework injection.

Key Vault References

Reference secrets stored in Azure Key Vault using @Microsoft.KeyVault(SecretUri=...) syntax.

Slot-Specific Settings

Mark settings as **slot-sticky** so they don't swap with the deployment slot. Essential for environment-specific values.

```
# Set app setting via CLI
az webapp config appsettings set \
  --name myApp \
  --resource-group myRG \
  --settings ENV=production \
  DB_HOST=prod-db.database.windows.net

# Make a setting slot-sticky
az webapp config appsettings set \
  --slot-settings ENV=staging
```

App Service Networking & Security

Built-In Security Features

- **Custom Domains & TLS/SSL:** Free managed certificates via App Service Managed Certificate
- **Authentication / Authorization:** Built-in Easy Auth with Azure AD, Google, Facebook, GitHub
- **IP Restrictions:** Allowlist/blocklist by IP range or service tag
- **VNet Integration:** Route outbound traffic through your Virtual Network (Standard+)
- **Managed Identity:** Authenticate to other Azure services without credentials in code

Monitoring App Service Applications

Application Insights

Full APM: distributed traces, dependency tracking, exceptions, live metrics. Attach with one click.

Diagnostic Logs

Enable HTTP, application, and detailed error logs. Stream to Log Analytics or download via FTP.

Log Stream

Real-time log tail in the Portal or CLI: `az webapp log tail`. Instant feedback during debugging.

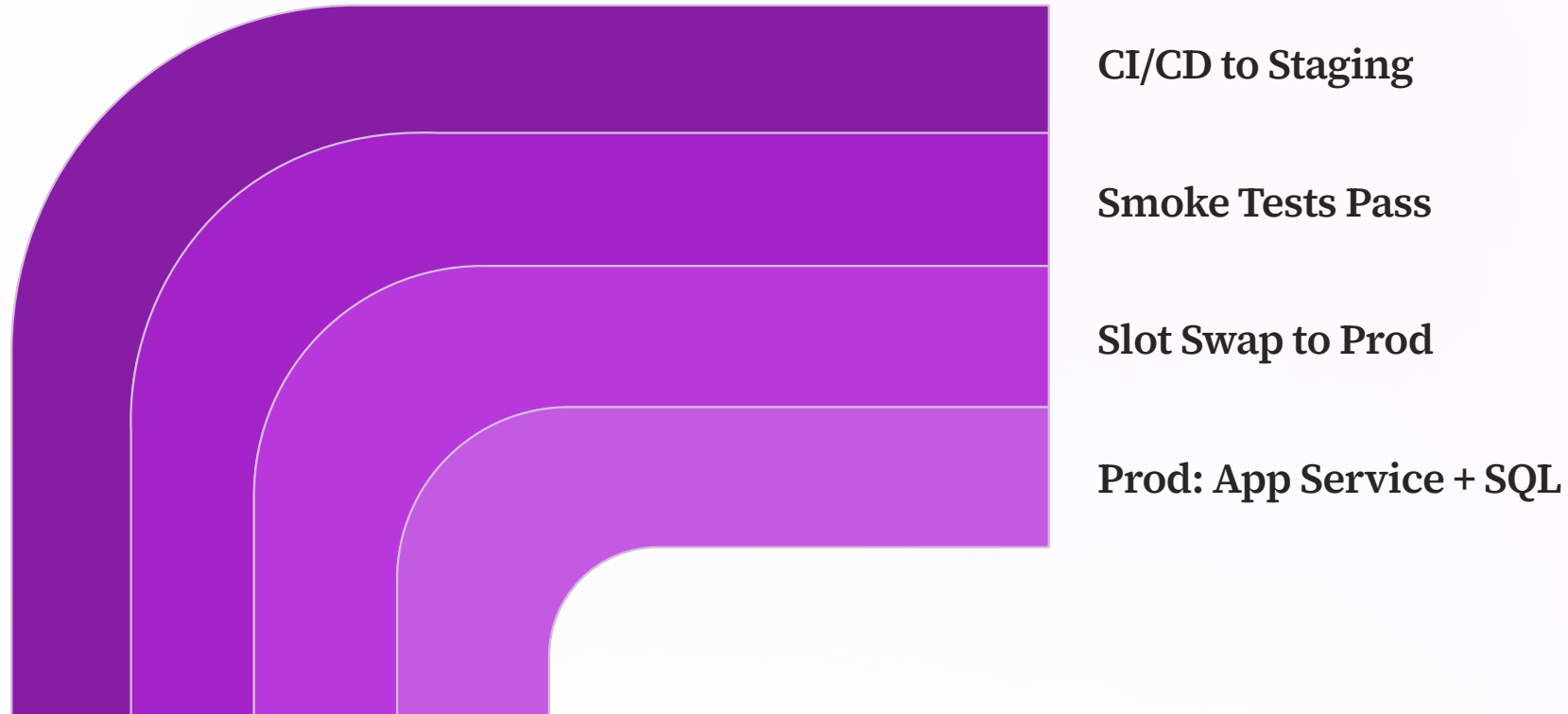
Health Checks

Configure a health endpoint. App Service replaces unhealthy instances automatically.

Azure Monitor Alerts

Set metric-based alerts (CPU > 80%, HTTP 5xx > threshold) with email or webhook notifications.

App Service — Architecture Summary



This pattern — CI/CD to staging, validate, swap to production — is the standard deployment workflow for App Service in enterprise environments.

DAY 1 LABS

Hands-On Labs — Day 1

Four guided labs to reinforce today's concepts through real Azure deployments.

Day 1 Lab List

1

Azure Setup for Developers

Configure your subscription, install Azure CLI, and authenticate your development environment.

2

Deploy Web App to App Service

Create an App Service Plan and deploy a web application using the Azure CLI and ZIP deploy.

3

Configure App Service

Set application settings, connection strings, and explore slot-sticky configuration values.

4

Scaling Web Apps

Configure manual and auto-scale rules, then observe instance behavior under simulated load.



Lab instructions are in your course lab guide. Raise your hand if you need help — your instructor is here to assist.

Day 1 — Key Takeaways



ARM is the control plane

All Azure management — Portal, CLI, SDK — goes through Azure Resource Manager.



Resource Groups define lifecycle

Group resources by application and environment. Deleting a group deletes everything inside.



App Service is your default web compute

PaaS, fully managed, supports auto-scale, slots, and built-in security. Start here for web apps.



Deployment slots enable zero-downtime releases

Deploy to staging, validate, then swap — instant rollback always available.

Coming Up — Day 2 Preview

Day 2 Topics

- Azure Functions — deep dive
- Durable Functions & orchestration patterns
- Azure Storage: Blob, Queue, Table
- Azure Cosmos DB fundamentals
- Connecting compute to data services

Before Tomorrow

Complete today's labs, especially **Deploy Web App to App Service** — Day 2 builds on this foundation.

Review the Azure Functions documentation overview linked in the course portal.



Questions? Reach out via the course Teams channel or flag your instructor before the session ends.

You're on Your Way to AZ-204

Day 1 complete. You've laid the foundation — from ARM and resource management to deploying and scaling production-grade web applications on App Service.

4

Core Topics

Covered today

4

Hands-On Labs

To complete today

200+

Azure Services

Available to explore

1

Certification

AZ-204 within reach



Microsoft Certified: Azure Developer Associate